Лабораторная работа №3

Verilog

Инструментарий и требования к работе

ПО	icarus verilog version 12.0(devel или stable) и новее
Установка	https://skkv-itmo.gitbook.io/comp-arch-cookies/literature/verilog
Справка	https://skkv-itmo.gitbook.io/comp-arch-cookies/verilog/start

Задание

Необходимо смоделировать работу кэша процессора на языке описания схем SystemVerilog. Кэш должен уметь вести и выдавать статистику. Кроме модуля кэша необходимо реализовать тестовый модуль `testbench`, который в конце запрашивает у кэша статистику и выводит её в поток вывода через `display` число попаданий, число обращений к кэшу и их процентное соотношение в формате:

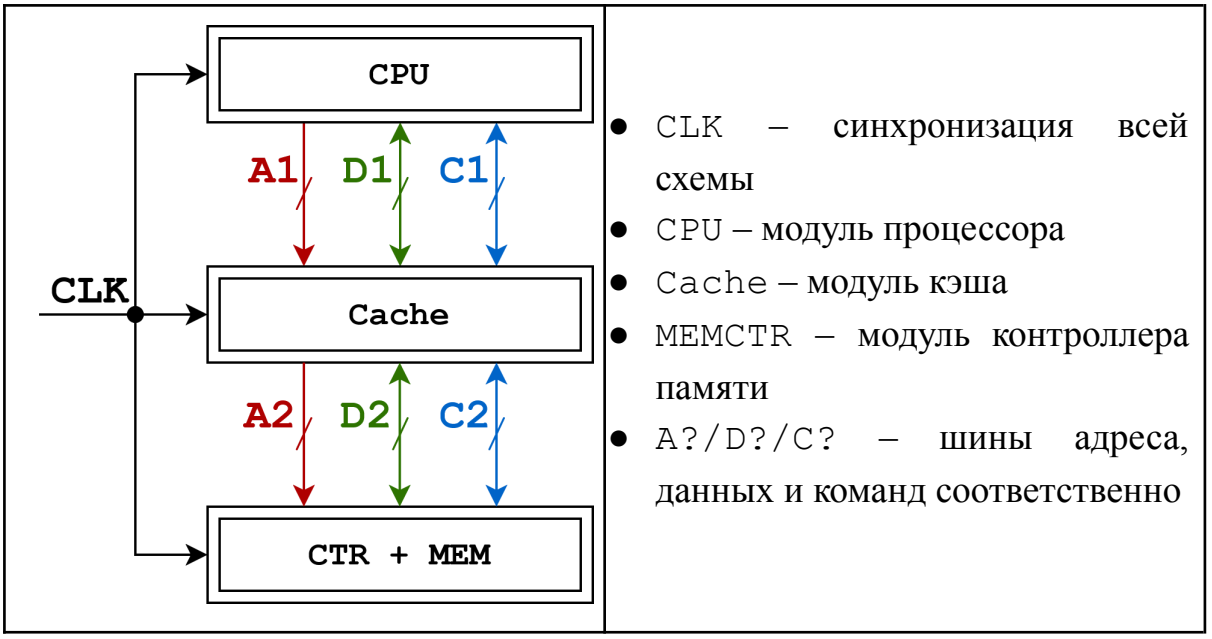
```
"| replacement |  hitrate  |    hits    |  accesses  |\n| :---------- |
--------: | ---------: | ----------: |\n| LRU         |  %3.4f%% | %10d |
%10d |\n| bpLRU       |  %3.4f%% | %10d | %10d |\n"
```

В случае, когда не было обращений к памяти, следует выводить `nan%`. Если какая-то из политик вытеснения не реализована, то выводится `-`.

Расхождение даже в один пробельный символ или потерянный знак `%` приведёт к тому, что ответ не будет зачтён. Допустимо отклонение в 0.0001% от ожидаемого ответа.

Все файлы с кодом на SystemVerilog должны быть в корне репозитория.

Модель взаимодействия модуля кэша с окружающим миром



- `CLK` – синхронизация всей схемы
- `CPU` – модуль процессора
- `Cache` – модуль кэша
- `MEMCTR` – модуль контроллера памяти
- `A?/D?/C?` – шины адреса, данных и команд соответственно

Не запрещено добавлять свои сигналы (например, `RESET` для перезапуска модуля).

Начальное состояние: кэш пуст, все кэш-линии в состоянии invalid.

Все приведённые ниже переменные/константы/параметры должны в явном виде располагаться в коде (желательно в одном файле). Не запрещено оставлять комментарий как был посчитан тот или иной параметр.

Параметр	**Пояснение**
`ADDRESS_LEN`	длина адреса (в битах)
`TAG_LEN`	длина тэга адреса (в битах)
`INDEX_LEN`	длина индекса блока кэш-линий (в битах)
`OFFSET_LEN`	длина смещения внутри кэш-линии (в битах)
`CACHE_SIZE`	размер кэша без учёта служебной инф-и (в байтах)
`CACHE_LINE_SIZE`	размер кэш-линии (в байтах)
`CACHE_LINE_COUNT`	кол-во кэш-линий
`CACHE_SET_COUNT`	кол-во блоков кэш-линий

CACHE_WAY	ассоциативность
MEMORY_SIZE	размер памяти (в байтах)

Интерпретация адреса кэшем (слева старшие биты, справа – младшие):

TAG	INDEX	OFFSET

Обмен данными по шинам

Команды и ответы на них передаются по шинам за несколько тактов подряд. Но между командой и ответом может быть произвольное кол-во тактов бездействия.

По шине A1 адрес передаётся за 2 такта: в первый такт TAG+INDEX, во второй – OFFSET. По шине A2 передаётся адреса без части OFFSET за 1 такт.

По шинам D (D1 и D2) в каждый такт передаётся по 16 бит данных, начиная с младших, little endian.

На линиях команд C (C1 и C2) значение держится всё время передачи команды или ответа.

В начальный момент времени (или после RESET) шиной 1 владеет CPU, а шиной 2 – Cache. После подачи команды и до окончания отправки ответа владение шиной переходит к Cache и MEMCTR соответственно. Владение шиной означает какое устройство задает логические уровни на проводах шины.

Параметры системы

Ниже указаны варианты конфигурации. Часть данных – общая для всех, часть – вариативна. Номер варианта каждого студента указан в таблице курса. Пустые ячейки – параметр необходимо вычислить самостоятельно.

По реализации должно быть однозначно понятно, что кэш реализован нужной конфигурации!

Общие параметры

Команды, запрашивающие несколько байт, не могут пересекать кэш-линию.

<table>
<tr><th colspan="4">CPU</th></tr>
<tr><td rowspan="2">Команды</td><td>CPU → Cache</td><td>C1_NOP
C1_INVALIDATE
C1_READ_8
C1_WRITE_8
C1_READ_16
C1_WRITE_16
C1_READ_32
C1_WRITE_32</td><td rowspan="2">Команда INVALIDATE означает инвалидацию всей кэш-линии, содержащей указанный адрес.

Число в командах означает кол-во бит, запрашиваемое данной командой.

NOP – no operation.
RESPONSE – ответ на команду.</td></tr>
<tr><td>CPU ← Cache</td><td>C1_NOP
C1_RESPONSE</td></tr>
</table>

<table>
<tr><th colspan="4">Cache</th></tr>
<tr><td>Конфигурация</td><td colspan="3">look-through write-back write-allocate</td></tr>
<tr><td>Политика вытеснения</td><td colspan="3">LRU и bit-pLRU</td></tr>
<tr><td rowspan="2">Команды</td><td>Cache → Mem</td><td>C2_NOP
C2_READ
C2_WRITE</td><td rowspan="2">Команды пишут и читают порциями, равными размеру кэш-линии.</td></tr>
<tr><td>Cache ← Mem</td><td>C2_NOP
C2_RESPONSE</td></tr>
</table>

Шина	**Обозначение**	**Размерность**
A1, A2	ADDR1_BUS_LEN, ADDR2_BUS_LEN	вычислить самостоятельно
D1, D2	DATA1_BUS_LEN, DATA2_BUS_LEN	16 бит
C1, C2	CTR1_BUS_LEN, CTR2_BUS_LEN	вычислить самостоятельно

Вариативная часть

Параметры	**Вариант 1**	**Вариант 2**	**Вариант 3**
ADDRESS_LEN		17	
TAG_LEN	8		
INDEX_LEN		4	4

OFFSET_LEN			5
CACHE_SIZE			2 Кбайта
CACHE_LINE_SIZE		64	
CACHE_LINE_COUNT		64	
CACHE_SET_COUNT	32		
CACHE_WAY	4		
MEMORY_SIZE	256 Кбайт		512 Кбайт

Задача

Необходимо смоделировать обращения к памяти по данным, которые будут происходить при выполнении следующего кода:

```
#define M 64
#define N 60
#define K 32
int8 a[M][K];
int16 b[K][N];
int32 c[M][N];
void mmul()
{
  int8 *pa = a;
  int32 *pc = c;
  for (int y = 0; y < M; y++)
  {
    for (int x = 0; x < N; x++)
    {
      int16 *pb = b;
      int32 s = 0;
      for (int k = 0; k < K; k++)
      {
        s += pa[k] * pb[x];
        pb += N;
      }
      pc[x] = s;
    }
    pa += K;
    pc += N;
  }
}
```

Массивы последовательно хранятся в памяти, и первый из них начинается с адреса 0x128.

Все локальные переменные и промежуточные значения вычислений лежат в регистрах процессора.

Тестирование в Actions

Все следующие команды производятся из корня репозитория.

1. Компиляция скрипта производится следующей командой:

```
iverilog -g2012 -o tb.out tb.sv
```

2. Запуск симуляции:

```
vvp tb.out
```